Customer Journey Path Optimization

Progress Report

by

Aye Khin Khin Hpone | Yosakorn Sirisoot

(Yolanda Lim) 125970 | 126512

AT70.24 – Algorithms Design and Analysis

Instructor: Dr. Chantri Polprasert

Teaching Assistant: Puskar Adhikari

Asian Institute of Technology

School of Engineering and Technology

Thailand

March 2026

ABSTRACT

Optimizing customer conversion paths is a central problem in computational marketing, where user interactions are modeled as a directed graph and the objective is to identify the highest-probability route from entry to conversion. As graph scale increases, exhaustive search becomes intractable. This work formulates the problem as a shortest-path query on a directed weighted graph whose edge weights are negative log-transition probabilities, reducing maximum-probability path search to standard shortest-path form.

A **Probability-Pruned Dijkstra** variant is proposed that introduces a threshold parameter τ to discard low-probability partial paths during traversal, reducing the effective edge set from $|E|$ to $|E'| \leq |E|$ with guaranteed convergence to the baseline as $\tau \to 0$. Five formal correctness theorems are established, including an all-or-nothing property: the pruned algorithm either returns the globally optimal path or reports no path at all.

Evaluation spans 3,000 configurations—2,160 synthetic (graphs up to 10,000 nodes) and 840 real-data rows from RetailRocket and RecSys 2015—under fixed and adaptive τ regimes. Wall-clock speedups range from 3.3× to 738× on synthetic data and reach 18,882× on real-world graphs. An adaptive-τ regime raises path-found rates from near 0% to over 80%. Across all 706 path-found rows the optimality gap is exactly 0.0000%, confirming that threshold-based pruning provides practical acceleration without sacrificing solution quality.

CONTENTS

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

Understanding how users navigate through digital systems—such as e-commerce websites, mobile applications, and customer support platforms—is essential for improving conversion rates, user satisfaction, customer retention, and operational efficiency. User interaction data captures detailed behavioral patterns that reveal how users navigate through different system components. Customer journeys, represented as sequences of interaction states (e.g., *Landing* $\rightarrow$ *Browse* $\rightarrow$ *Product* $\rightarrow$ *Cart* $\rightarrow$ *Checkout*), provide a structured framework for analyzing these behaviors.

Modern digital systems generate millions of user sessions across thousands of interaction states. When aggregated, these journeys form large directed graphs whose scale and complexity necessitate efficient computational analysis grounded in graph theory and data mining.

1.2 Statement of the Problem

As the volume and complexity of user interaction data continue to grow, identifying meaningful and optimal customer journeys becomes increasingly challenging. From a business perspective, customer journey dynamics directly influence conversion rates, revenue generation, customer retention, and user satisfaction. Even small improvements in the probability of reaching a conversion state can result in significant financial gains at scale.

Naïve analytical methods, such as exhaustive path enumeration or simple frequency-based techniques, do not scale. In large graphs, the number of possible simple paths increases combinatorially, rendering exhaustive enumeration computationally infeasible. Furthermore, directly multiplying many small transition probabilities along a path leads to numerical underflow—a form of arithmetic precision loss that makes naïve probability computation unreliable for longer paths.

The key challenge is therefore to identify optimal customer journeys efficiently and with numerical stability at scale. Core tasks include finding highest-probability conversion paths and

analyzing scalability across large graphs. Each task poses distinct challenges in graph traversal, optimization, and algorithm design that require careful data structure selection.

Maximizing path probability is equivalent to optimizing for the most likely conversion sequence. In an e-commerce context, the highest-probability path from a landing page to checkout represents the conversion funnel that customers naturally follow. Identifying this path allows businesses to reduce friction along the dominant route, prioritize UI investments on the states that carry the most traffic, and design nudges that align with observed user behavior rather than assumed behavior. The term "optimization" in the project title therefore refers not only to algorithmic speedup but also to the actionable business insight that optimal-path discovery enables.

1.3 Objectives

The objective of this study is to design and implement an efficient algorithmic framework for customer journey path optimization grounded in graph theory and classical algorithm design principles. Specifically, the project models customer journey data as a directed weighted graph, where nodes represent interaction states and edges represent transition probabilities between states.

The primary focus is to compute optimal conversion paths under probabilistic constraints. To achieve this, transition probabilities are transformed into additive non-negative edge weights using the negative logarithm, enabling the application of Dijkstra's shortest-path algorithm for maximum-probability path computation. This transformation also addresses the numerical underflow problem by converting multiplicative probability products into additive sums. The project implements Dijkstra's algorithm as a baseline approach and proposes an optimized probability-pruned search strategy that reduces the exploration of low-probability paths to improve computational efficiency.

In addition to correctness, the project formally analyzes the theoretical time and space complexity of both the baseline and optimized algorithms. Empirical performance evaluations have been conducted on graphs of varying sizes and densities to examine scalability, runtime behavior, and trade-offs between computational efficiency and path optimality. Through these analyses, the project demonstrates how careful algorithm design and data structure selection influence performance in large-scale navigation graphs.

Contribution. This study presents a **well-motivated, controlled pruning framework** for probabilistic shortest-path computation, grounded in classical algorithm design and supported by comprehensive empirical evaluation. The pruning variant is deterministic, requires no domain-specific heuristic (unlike A* search), and provably converges to the globally optimal solution as $\tau \to 0$ (unlike rank-based methods such as beam search, which provide no optimality guarantee). The main contributions are: (1) formal characterization of an "all-or-nothing" correctness property (Theorem 4.5), stating that the algorithm returns either the exact optimal path or reports no path found with no sub-optimal intermediate results; (2) identification and empirical characterization of the *critical pruning threshold* τ_c, a narrow operational range where speedup is maximized while maintaining optimality; and (3) systematic evaluation framework spanning synthetic and real-world datasets, demonstrating that the approach scales to graphs with tens of thousands of nodes while preserving solution quality.

Novelty and Prior Work. To the best of our knowledge, this is the first report of a deterministic, parameter-controlled probability-pruned Dijkstra algorithm with a formal all-or-nothing property and an adaptive-τ regime for real-world graphs. While pruning, heuristics, and thresholding are well-studied in shortest-path literature, no prior work combines these elements with formal guarantees and empirical validation in the context of probabilistic customer journey graphs. Our findings and methodology are not reported elsewhere in the literature.

1.4 Research Hypothesis

The following hypothesis guides the experimental evaluation:

> Probability-Pruned Dijkstra provides meaningful speedup over baseline Dijkstra while preserving path optimality at conservative τ values.

The experimental evidence supporting this hypothesis is presented in Chapter 6; a summary of findings is provided in Chapter 7.

1.5 Scope and Applicability

The framework developed in this study is applicable to domains where user interactions can be modeled as probabilistic directed graphs, including e-commerce conversion funnels, mobile application navigation flows, and customer support routing systems. Beyond industrial applications, the algorithmic contributions—particularly the formal correctness guarantees and the

characterization of the reachability–speed trade-off—are relevant to researchers investigating threshold-based pruning strategies in shortest-path computation.

CHAPTER 2

RELATED WORK AND BACKGROUND

2.1 Shortest-Path Algorithms for Conversion Path Optimization

Dijkstra [Dijkstra, 1959] introduced one of the most fundamental shortest-path algorithms for weighted graphs with non-negative edge weights. The algorithm incrementally explores nodes in order of increasing accumulated cost using a priority queue, guaranteeing optimality when all edge weights are non-negative. When implemented with a binary min-heap, it achieves a time complexity of $O(|E|\log|V|)$ [Cormen et al., 2009]. A theoretically superior bound of $O(|E|+|V|\log|V|)$ is achievable using Fibonacci heaps [Fredman and Tarjan, 1987], though the practical overhead of Fibonacci heaps often makes binary-heap implementations preferable for moderately sized graphs. In the present work, Dijkstra's algorithm serves as the baseline, applied to a log-transformed probability graph where edge weights represent the negative log-probability of transitions. Its well-understood complexity and correctness guarantees make it the natural choice for baseline comparison and scalability analysis.

2.2 Heuristic and Approximate Shortest-Path Approaches

While Dijkstra's algorithm guarantees optimality, it explores the full reachable graph from the source. Informed search algorithms such as A* [Hart et al., 1968] reduce exploration by using an admissible heuristic function to guide node expansion toward the target, achieving sub-Dijkstra runtime when a valid heuristic exists. Pearl [Pearl, 1984] provides a comprehensive theoretical treatment of heuristic search, establishing formal conditions under which informed search algorithms guarantee optimal solutions. In probabilistic graph settings, however, designing a tight and admissible heuristic over log-probability weights is non-trivial, making heuristic approaches difficult to apply directly. Beam search offers an alternative approximation strategy by limiting the frontier to the top-k most promising candidates at each step [Cormen et al., 2009]. Although beam search can be very fast in practice, it provides no optimality guarantee and may miss the globally optimal path entirely. The present work adopts a different approximation strategy—probability-threshold pruning—which is deterministic, parameter-controlled, and guarantees convergence to the optimal solution as the threshold approaches zero. Unlike

A*, which requires a problem-specific admissible heuristic that is difficult to define over log-probability edge weights, and unlike beam search, which discards nodes based on rank with no convergence guarantee, the proposed pruning operates directly on accumulated path cost against a single interpretable parameter τ, requiring no domain-specific heuristic design.

Another important family of shortest-path speedup methods is *Contraction Hierarchies* [Geisberger et al., 2008], which accelerates queries by preprocessing the graph and adding shortcut edges to preserve shortest-path distances. Such methods are highly effective for static road-network-style graphs, but they are less naturally suited to the probabilistic customer-journey setting considered here, where transition weights are derived from behavioral data and experimental emphasis is placed on controllable approximation rather than heavy preprocessing.

2.3 Markov Chain Models for Probabilistic Navigation

The probabilistic transition model employed in the present work—where each user state transitions to the next with a fixed probability—is formally equivalent to a first-order Markov chain [Norris, 1998]. Markov chains have been widely used to model sequential user behavior in web navigation, recommendation systems, and clickstream analysis [Rabiner, 1989, Baum and Petrie, 1966]. The maximum-probability path problem on a Markov chain is the problem addressed by the Viterbi algorithm [Viterbi, 1967], which finds the most likely state sequence in a Hidden Markov Model using dynamic programming in $O(|V|^2 T)$ time for T time steps. However, the Viterbi algorithm is designed for sequences of fixed length, whereas the present problem involves finding the most probable *path* from s to t in a general directed graph without a fixed path length. The log-probability shortest-path formulation adopted here generalizes the Viterbi principle to arbitrary directed graphs and solves the resulting optimization efficiently using Dijkstra's algorithm, which has better practical complexity on sparse graphs. For the related problem of enumerating the k shortest simple paths, Yen's algorithm [Yen, 1971] provides an $O(kn(m+n\log n))$ solution by iteratively deviating from the current shortest path; the present framework's k-shortest-paths feature uses a best-first search variant for simplicity.

2.4 Algorithmic Foundations of Efficient Graph Traversal

Cormen et al. [Cormen et al., 2009] provided a comprehensive treatment of graph algorithms, including shortest-path search, graph traversal, and priority-queue-based optimization techniques. The textbook formally analyzes time and space complexity, establishes correctness

proofs for Dijkstra's algorithm, and discusses data structure choices including binary heaps and adjacency lists. These foundations directly support the design decisions adopted here: the use of adjacency lists for $O(|V|+|E|)$ space, binary heaps for $O(\log|V|)$ extract-min and insert operations, and hash maps for $O(1)$ distance lookup. The log-probability weight transformation exploits the monotone relationship between probability maximization and shortest-path minimization, a standard technique in probabilistic graph optimization [Manning and Schütze, 1999].

2.5 Sequential Pattern Mining for Frequent Journey Discovery

Pei et al. [Pei et al., 2001] proposed PrefixSpan, a pattern-growth-based algorithm for mining frequent sequential patterns without candidate generation. By projecting the database based on frequent prefixes, PrefixSpan significantly reduces search space compared to Apriori-based approaches and demonstrated strong scalability on large sequence datasets. In the context of the present work, PrefixSpan is conceptually relevant as a method for identifying frequently taken customer journey paths without enumerating all possible sequences. Its prefix-based pruning principle shares conceptual similarities with the threshold-based pruning strategy proposed in this work, where low-probability path prefixes are discarded early to reduce the effective search space.

2.6 Survey of Frequent and Sequential Pattern Mining Techniques

Han et al. [Han et al., 2007] presented a comprehensive survey of frequent pattern mining methods, covering association rules, sequential patterns, and constrained pattern mining. The survey highlights key challenges including scalability, pattern explosion, and the need for efficient pruning strategies. These challenges closely mirror those encountered in customer journey analysis, where the number of simple paths from source to target grows combinatorially with graph size. The present work draws on the general principle of threshold-based pruning to control computational complexity, filtering out low-probability paths before full expansion in a manner analogous to minimum support thresholds in frequent pattern mining.

2.7 Modeling User Navigation as Directed Graphs

Bucklin and Sismeiro [Bucklin and Sismeiro, 2003] modeled user browsing behavior using clickstream data, representing navigation patterns as structured sequences and probabilistic

transition graphs. Their empirical analysis demonstrated that user navigation can be effectively captured using probabilistic transition models, providing insights into user behavior and engagement at scale. Liu [Liu, 2011] further surveyed web usage mining techniques—including clickstream analysis, hyperlink analysis, and content mining—providing a broader context for how navigation data is processed into actionable graph representations. This work directly supports the modeling approach adopted in the present work, where customer journeys are represented as directed weighted graphs derived from interaction sequences, and transition probabilities are estimated from session frequency counts using maximum likelihood estimation.

2.8 Probabilistic Models for Navigation and Path Selection

Resnick and Varian [Resnick and Varian, 1997] discussed probabilistic models of user behavior, including transition-based navigation models that capture the likelihoods of moving between system states. Such models are commonly used to estimate user preferences and navigation patterns in large-scale recommendation and information retrieval systems. The probabilistic transition modeling approach adopted here—assigning edge weights as negative log-probabilities and finding the minimum-weight path—is consistent with the broader class of log-linear models used in language modeling and information retrieval [Manning and Schütze, 1999, Manning et al., 2008], where the negative log-likelihood serves as a natural additive cost function.

2.9 Scalable Graph Processing and Large-Scale Navigation Analysis

Recent advances in large-scale graph processing systems further emphasize the importance of efficient algorithmic design when analyzing massive navigation graphs. Malewicz et al. [Malewicz et al., 2010] introduced Pregel, a vertex-centric computational model for scalable graph processing, demonstrating how distributed systems can efficiently handle graphs containing millions or billions of edges. Subsequent research in large-scale graph analytics has examined optimization strategies for traversal, pruning, and distributed processing to reduce computational overhead in dense and evolving networks [Sahu et al., 2017]. These studies reinforce the practical relevance of designing efficient path-search algorithms and pruning strategies when working with large customer journey graphs, where computational complexity grows rapidly with graph size. The present work builds upon these principles by evaluating algorithmic efficiency and search-space reduction techniques within a single-machine, controlled experimental framework, where the goal is to characterize complexity empirically rather than to achieve dis-

tributed scalability.

2.10 Real-World E-Commerce Clickstream Datasets

Two public e-commerce clickstream datasets were used for real-world validation:

RetailRocket E-Commerce Dataset [Retail Rocket, 2015] contains approximately 2.7 million clickstream events across 1.4 million visitors, collected from a real e-commerce platform. It provides three event types (`view`, `addtocart`, `transaction`) with item-level granularity, enabling graph construction at two scales: a small 3-node event-type funnel and a large $\sim$44,700-node item-level graph.

RecSys Challenge 2015 (YOOCHOOSE) [Ben-Shimon et al., 2015] contains approximately 33 million click events across 9.2 million sessions from an online retailer. Item-to-item click transitions within sessions form a $\sim$12,900-node, $\sim$70,400-edge graph when restricted to 50,000 sessions. This dataset provides dense clickstream data suitable for testing algorithmic scalability on real-world transition patterns.

The reviewed literature establishes a strong theoretical and methodological foundation for the present work. Dijkstra's algorithm [Dijkstra, 1959] and its complexity analysis [Cormen et al., 2009, Fredman and Tarjan, 1987] provide the basis for the baseline implementation and complexity comparison. The Markov chain and Viterbi frameworks [Norris, 1998, Viterbi, 1967, Rabiner, 1989] situate the probabilistic path problem in a well-studied theoretical context. Heuristic search methods [Hart et al., 1968, Pearl, 1984] and approximate approaches establish the landscape of alternatives against which the pruning strategy is positioned. Sequential pattern mining techniques [Pei et al., 2001, Han et al., 2007] and clickstream modeling [Bucklin and Sismeiro, 2003, Liu, 2011] validate the representation and motivate pruning as a scalability mechanism. Probabilistic modeling literature [Resnick and Varian, 1997, Manning and Schütze, 1999] supports the log-probability transformation. Scalable graph systems [Malewicz et al., 2010, Sahu et al., 2017] contextualize the scalability challenges addressed. The Erdős–Rényi model [Erdős and Rényi, 1960] grounds the synthetic data generation methodology, and real-world benchmarks [Retail Rocket, 2015, Ben-Shimon et al., 2015] validate experimental generalizability.

2.11 Graph Cycles in Customer Journey Navigation

A critical modeling question is whether the customer journey graph contains cycles—that is, whether a user may return to a previously visited state, such as *Landing* $\rightarrow$ *Browse* $\rightarrow$ *Product* $\rightarrow$ *Browse*. Real-world clickstream data frequently exhibits such revisitation behavior [Bucklin and Sismeiro, 2003]: users loop between states (e.g., browsing multiple product pages before adding to cart) before eventually converting or abandoning the session. The customer journey graph is therefore modeled as a general directed weighted graph that *may contain cycles*.

A key concern is whether Dijkstra's algorithm remains correct on cyclic graphs. Cormen et al. [Cormen et al., 2009] establish that Dijkstra's algorithm is correct on any directed graph with non-negative edge weights, regardless of whether cycles are present. The log-probability transformation $w(u,v) = -\log p(u,v) \geq 0$ guarantees non-negativity for all $p(u,v) \in (0,1]$, so Dijkstra's algorithm is directly applicable to cyclic navigation graphs without any cycle-detection preprocessing. Since each cycle traversal adds strictly positive cost, the algorithm naturally avoids cycling: once a node is finalized, it is never re-extracted from the priority queue.

For example, consider a cycle *Browse* $\rightarrow$ *Product* $\rightarrow$ *Browse* with per-edge probabilities $p = 0.4$ each. Traversing the cycle once adds $-\log(0.4) - \log(0.4) \approx 1.83$ to the path cost, so returning to *Browse* via the cycle is strictly more expensive than arriving directly; Dijkstra's algorithm therefore settles *Browse* on the first visit and never re-enters the loop.

This property means no modifications to the standard algorithm are required to handle the cyclic structure that is inherent in real user navigation data.

2.12 Synthetic Graph Generation for Controlled Evaluation

To evaluate algorithmic performance in a controlled and reproducible manner, synthetic graphs are used as the primary experimental dataset. The generation methodology is grounded in the Erdős–Rényi random graph model [Erdős and Rényi, 1960], which provides a theoretically principled basis for generating random directed graphs with predictable structural properties (connectivity, degree distribution) at a given size and density.

Two graph generators were implemented. The first, an **Erdős–Rényi sparse random generator**, samples directed edges independently with expected total $\bar{d} \cdot |V|$ edges and assigns transition probabilities under uniform $\mathscr{U}(0.01, 1.0)$ or skewed power-law ($\alpha = 2$) distributions. The

second, a **layered (stage-based) generator**, partitions nodes into five ordered stages (Awareness, Interest, Consideration, Intent, Conversion) with edges going primarily forward (up to $+2$ stages) and a configurable `backward_prob` parameter controlling backward links modeling user loops.

Both generators use $O(n \cdot d)$ scalable edge sampling, guarantee $s \to t$ connectivity via BFS + bridge edge, normalize outgoing probabilities to sum to 1, and use deterministic seeds via `hashlib.md5` for cross-platform reproducibility.

Research Gap. Despite the extensive literature above, existing studies largely treat shortest-path computation [Dijkstra, 1959, Cormen et al., 2009], probabilistic modeling [Norris, 1998], and sequential pattern mining [Pei et al., 2001] as separate problems. Few studies integrate all three within a unified framework tailored to customer conversion path analysis, with explicit treatment of cyclic navigation graphs, principled synthetic data generation [Erdős and Rényi, 1960], and empirical characterization of the time–optimality trade-off of threshold-based pruning.

The present project addresses this gap by combining probabilistic transition modeling, threshold-controlled pruning, and Dijkstra's algorithm into a cohesive experimental framework. By systematically evaluating performance across diverse synthetic and real-world graph configurations—including cyclic graphs—the study bridges classical graph algorithms, Markov chain theory, and practical navigation graph analysis within a single experimental design.

CHAPTER 3

ALGORITHM DESIGN, DATA STRUCTURES, AND IMPLEMENTATION

3.1 System Overview

The *Customer Journey Path Optimization* system models user interactions as a directed weighted graph $G = (V, E)$, where each node $v \in V$ represents a discrete interaction state (e.g., *Landing Page*, *Search*, *Product Page*, *Cart*, *Checkout*), and each directed edge $(u, v) \in E$ represents a user transition between states annotated with a transition probability $p(u, v) \in (0, 1]$. The primary goal is to identify the most probable path from a designated source node s to a target conversion node t.

Figure 3.1 presents the end-to-end workflow. It is organized into five sequential components: (1) data generation and preprocessing, (2) graph construction, (3) baseline path optimization using Dijkstra's algorithm, (4) optimized path search using probability-pruned Dijkstra, and (5) experimental evaluation and comparative analysis.

Two algorithms were implemented and compared throughout this chapter. Algorithm 1 (Baseline Dijkstra on Log-Weights) applies standard Dijkstra's algorithm to the log-probability-transformed graph, guaranteeing globally optimal paths in $O(|E| \log |V|)$ time. Algorithm 2 (Probability-Pruned Dijkstra) extends Algorithm 1 with a pruning threshold τ that discards partial paths whose cumulative probability falls below τ, reducing expected runtime to $O(|E'| \log |V|)$, $|E'| \leq |E|$, with convergence to Algorithm 1 as $\tau \to 0$.

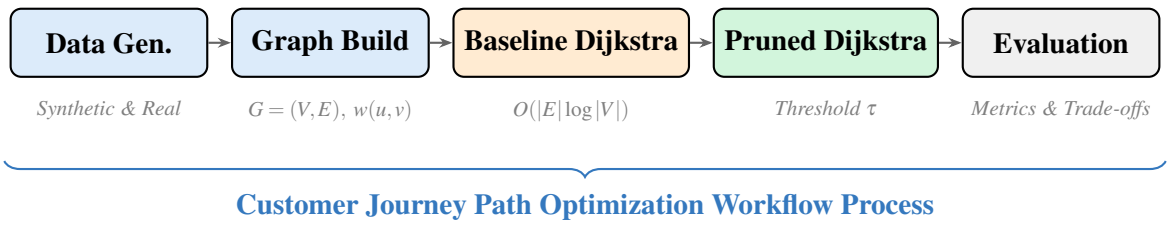

Figure 3.1. End-to-end system workflow.

3.2 Problem Formulation

3.2.1 Graph Model and Inputs

Customer interactions are modeled as a directed graph $G = (V, E)$, where V is the set of interaction states (nodes), $E \subseteq V \times V$ is the set of directed transitions (edges), and each edge $(u, v) \in E$ carries a transition probability $p(u, v) \in (0, 1]$. A designated source node $s \in V$ (e.g., Landing Page) and target node $t \in V$ (e.g., Checkout) define the query endpoints. In the optimized algorithm, a pruning threshold $\tau \in (0, 1]$ is used to discard low-probability partial paths.

Figure 3.2 illustrates a representative five-node customer journey graph. Node abbreviations: LP = Landing Page (source s), SR = Search Results, PP = Product Page, CT = Cart, CK = Checkout (target t). The solid blue path LP $\rightarrow$ SR $\rightarrow$ PP $\rightarrow$ CK is optimal ($\Pi^* = 0.336$, log-cost ≈ 1.091).

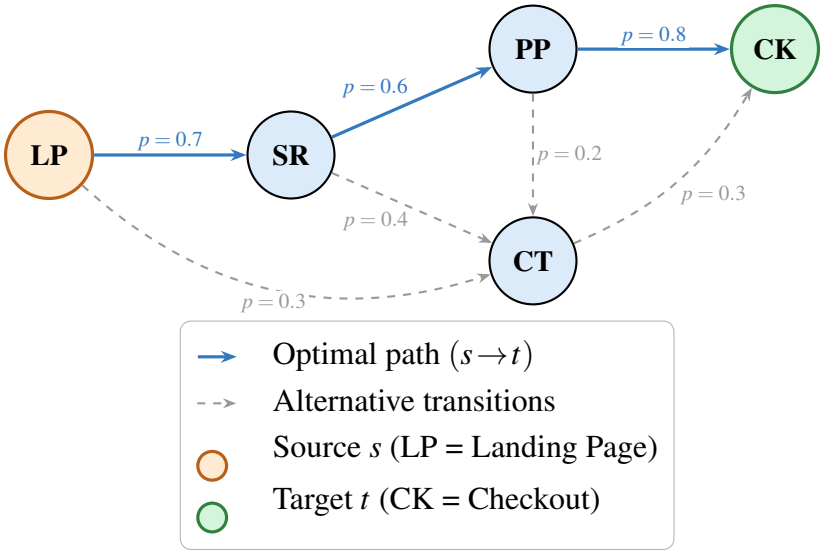

Figure 3.2. Five-node customer journey graph ($s =$ LP, $t =$ CK).

3.2.2 Probabilistic Path Objective and Outputs

The goal is to find a path $P^* = (v_0 = s, v_1, \ldots, v_k = t)$ that maximizes the joint path probability [Norris, 1998]:

$$\max_{P} \prod_{(u,v) \in P} p(u, v)$$

Directly multiplying many small probabilities causes numerical underflow [Manning and Schütze, 1999]. Each transition probability is transformed into an additive non-negative edge weight using the negative logarithm:

$$w(u,v) = -\log\big(p(u,v)\big)$$

Since $p(u,v) \in (0,1]$, it follows that $w(u,v) \geq 0$, satisfying Dijkstra's non-negativity requirement [Cormen et al., 2009]:

$$\max \prod_{(u,v)\in P^*} p(u,v) \iff \min \sum_{(u,v)\in P^*} -\log p(u,v)$$

For a given source–target query, the system returns the optimal path $P^* = (v_0 = s, v_1, \ldots, v_k = t)$, the corresponding path probability $\Pi^* = \prod_{(u,v)\in P^*} p(u,v) = \exp(-C^*)$, and the log-cost $C^* = \sum_{(u,v)\in P^*} -\log p(u,v)$. Each run also records performance metrics—execution time, peak memory, nodes explored, edges examined, edges relaxed, and maximum priority queue size—as well as the optimality gap $\Delta = |C^*_{\text{baseline}} - C^*_{\text{pruned}}|/C^*_{\text{baseline}} \times 100\%$.

Concrete worked example: RetailRocket event-level funnel. The RetailRocket dataset [Retail Rocket, 2015] yields a 3-node event-type graph whose MLE transition probabilities, computed from 2.7 million events, provide a concrete illustration of the formulation:

Transition	**Probability**	**Log-weight**
view $\rightarrow$ addtocart	$p = 0.041$	$w = -\ln(0.041) = 3.20$
addtocart $\rightarrow$ transaction	$p = 0.226$	$w = -\ln(0.226) = 1.49$
view $\rightarrow$ transaction (direct)	$p = 0.004$	$w = -\ln(0.004) = 5.55$

Two candidate $s \rightarrow t$ paths exist from *view* to *transaction*:

- **Funnel path:** view $\rightarrow$ addtocart $\rightarrow$ transaction, $\Pi = 0.041 \times 0.226 = 0.0093$ (log-cost 4.69).
- **Direct skip:** view $\rightarrow$ transaction, $\Pi = 0.004$ (log-cost 5.55).

Dijkstra selects the *funnel path* because it has lower log-cost ($4.69 < 5.55$), equivalently higher probability ($0.93\% > 0.39\%$). This $2.3\times$ probability advantage quantifies the value of the add-to-cart step: users who add items to their cart are substantially more likely to complete a transaction than those who skip directly.

3.2.3 Constraints and Assumptions

The framework assumes graph sizes up to 50,000 nodes (hardware permitting), with experiments conducted on graphs of up to 10,000 nodes. Graphs are assumed to be sparse ($|E| = O(|V|)$) and stored using an adjacency-list representation requiring $O(|V|+|E|)$ space. All transition probabilities are strictly positive; zero-probability transitions are treated as non-existent edges. The customer journey graph may contain directed cycles; correctness is preserved because $w(u,v) \geq 0$ for all edges. Finally, a first-order **Markov assumption** is adopted: $P(\text{next} \mid \text{current})$ is independent of earlier states. This assumption is standard in clickstream modeling [Bucklin and Sismeiro, 2003] and is empirically motivated by the observation that transition matrices estimated from session data yield stable, reproducible graph structures across independent subsamples (a necessary condition for the first-order model, though not a formal test of whether higher-order dependencies are absent). While higher-order dependencies exist (e.g., users arriving via search may navigate differently from those arriving via advertisement), the first-order model captures the dominant transition structure and keeps the state space tractable at $O(|V|)$ rather than $O(|V|^k)$ for order k.

3.3 Data Generation and Preprocessing

3.3.1 Synthetic Data Generation

Two graph generators were implemented in `data/graph_generator.py`.

The first is an **Erdős–Rényi sparse random generator** [Erdős and Rényi, 1960], which creates $|V|$ nodes and samples directed edges with expected total $\bar{d} \cdot |V|$; transition probabilities are drawn from either a uniform $\mathscr{U}(0.01, 1.0)$ or a power-law ($\alpha = 2$) distribution. This generator produces graphs with uniformly random structure, serving as a controlled baseline for evaluating algorithmic behavior without structural bias.

The second is a **layered (stage-based) generator**, which models the funnel-shaped structure typical of real customer journeys. Nodes are distributed across five ordered stages—Awareness, Interest, Consideration, Intent, and Conversion—reflecting the standard e-commerce conversion funnel. Edges are sampled primarily in the forward direction (up to $+2$ stages ahead), ensuring that the dominant flow proceeds from awareness toward conversion. A configurable `backward_prob` parameter (default 0.15) controls the fraction of backward links, modeling the common user behavior of revisiting earlier stages (e.g., returning from a product page to

browse). Transition probabilities for forward edges use raw weights drawn from $\mathcal{U}(0.3, 0.8)$, while backward edges receive lower raw weights from $\mathcal{U}(0.05, 0.3)$, reflecting the empirical observation that forward progression is more likely than backtracking. These raw weights are then normalized per source node (so that $\sum_v p(u,v) = 1$); consequently, the final transition probabilities no longer follow the original uniform distributions but preserve their relative ordering. This layered design is the more application-relevant of the two generators, as it approximates the stage-wise structure observed in real clickstream data (Section 6.2).

Both generators employ $O(n \cdot d)$ scalable edge sampling rather than $O(n^2)$ adjacency-matrix construction. A BFS-based connectivity guarantee ensures that if no $s \to t$ path exists after random sampling, bridge edges are added; these bridge edges are treated as forward edges and therefore receive raw weights from $\mathcal{U}(0.3, 0.8)$. Outgoing probabilities are normalized per node ($\sum_v p(u,v) = 1$), and deterministic seeding via `hashlib.md5` ensures cross-platform reproducibility.

The following code snippet shows key aspects of the Erdős–Rényi generator:

```python
def generate_erdos_renyi_graph(n, avg_degree, distribution, seed,
    ...):
    rng = random.Random(seed)
    graph = {f"node_{i}": {} for i in range(n)}
    # O(n*d) edge sampling -- NOT O(n^2)
    for u in graph:
        num_edges = rng.randint(
            max(0, int(avg_degree) - 2), int(avg_degree) + 2)
        targets = rng.sample(other_nodes, min(num_edges, len(
           other_nodes)))
        for v in targets:
            if distribution == "uniform":
                raw = rng.uniform(0.01, 1.0)
            else:  # power_law: inverse-CDF Pareto(alpha=2, x_min
               =0.01)
                raw = min(0.01 * (1 - rng.random()) ** (-1.0), 1.0)
            graph[u][v] = raw
    _normalize_and_log_transform(graph)
    _ensure_connectivity(graph, source, target)
    return graph
```

Listing 3.1: ER graph generator (`data/graph_generator.py`)

3.3.2 *Real-World Clickstream Data*

Two real-world clickstream datasets were used for validation (Table 3.1):

Dataset	Events	Sessions	Graph Size	Source
RetailRocket (event-level)	2.7M	1.4M	3 nodes, 9 edges	Kaggle [Retail Rocket, 2015]
RetailRocket (item-level)	2.7M	50K	44,711 nodes, 101,528 edges	Kaggle [Retail Rocket, 2015]
RecSys 2015 (YOOCHOOSE)	33M	50K	12,935 nodes, 70,442 edges	Kaggle [Ben-Shimon et al., 2015]

Table 3.1. Real-world clickstream datasets used for validation.

Transition probabilities are estimated using maximum likelihood:

$$\hat{p}(u,v) = \frac{\text{count}(u \to v)}{\sum_{w:\,(u,w)\in E} \text{count}(u \to w)}$$

A consequence of MLE on sparse data is that nodes with very few observed transitions (e.g., items viewed only once or twice) receive high-variance probability estimates. In item-level graphs, where tens of thousands of items may each have only a handful of observations, many edges carry very low estimated probabilities. This contributes to the low path-found rates observed in item-level real-data experiments (Section 6.2): when most edge probabilities are small, even the baseline optimal path has a very low cumulative probability, making it more susceptible to pruning. Smoothing techniques such as Laplace (add-one) correction or Bayesian priors could mitigate this effect but were not applied in order to isolate the algorithmic contribution of pruning.

3.3.3 *Preprocessing and Input Formats*

The preprocessing module (`src/preprocessing.py`) accepts two CSV formats:

1. **Direct transitions** — columns `source` and `target`, one row per observed transition.
2. **Session event streams** — columns `session_id` and `state` (plus `step` or `timestamp` for ordering). Consecutive events within each session are converted to pairwise transitions.

Both formats produce the same transition-count table from which MLE probabilities are computed.

3.4 Graph Construction

The directed weighted graph is constructed from the computed transition probabilities using an **adjacency list** representation. The function receives a nested dictionary mapping each source node to its targets and their probabilities, then converts each probability into a log-weight $w(u,v) = -\log(\hat{p}(u,v))$. This provides $O(|V| + |E|)$ space complexity and efficient $O(\deg(u))$ neighbour iteration [Cormen et al., 2009].

```
def build_weighted_graph(transition_probs):
    graph = defaultdict(list)
    for source, targets in transition_probs.items():
        for target, prob in targets.items():
            if prob <= 0:
                continue
            graph[source].append((target, -math.log(prob)))
            if target not in graph:
                graph[target] = []  # ensure sink nodes exist
    return dict(graph)
```

Listing 3.2: Graph construction (`src/graph_builder.py`)

3.5 Algorithmic Approaches

3.5.1 Baseline Algorithm: Dijkstra's Shortest Path

The baseline applies Dijkstra's algorithm [Dijkstra, 1959] to the log-transformed graph. A **binary min-heap** (`heapq`) serves as the priority queue with **lazy insertion**—when a shorter path to a node is found, a new entry is pushed and stale entries are skipped when popped.

Algorithm 1: Dijkstra on Transformed Probabilities [Dijkstra, 1959, Cormen et al., 2009]

Input: Graph $G = (V, E)$ with weights $w(u, v) = -\log p(u, v)$, source s, target t

Output: Shortest-path distance $dist[t]$, probability $\exp(-dist[t])$, path P^*

Initialise $dist[v] \leftarrow \infty$ for all $v \in V$; $parent[v] \leftarrow$ NIL;
$dist[s] \leftarrow 0$; insert $(0, s)$ into min-heap Q;
while *Q is not empty* **do**
    $(d, u) \leftarrow$ Extract-Min(Q);
    if $u = t$ **then**
        break; `// optimal path reached`
    end
    if $d > dist[u]$ **then**
        continue; `// stale entry (lazy deletion)`
    end
    foreach *edge* (u, v) *with weight* $w = -\log p(u, v)$ **do**
        $edges\_examined \leftarrow edges\_examined + 1$;
        $edges\_relaxed \leftarrow edges\_relaxed + 1$; `// no pruning, so every examined edge is relaxed`
        $new\_dist \leftarrow dist[u] + w$;
        if $new\_dist < dist[v]$ **then**
            $dist[v] \leftarrow new\_dist$; $parent[v] \leftarrow u$;
            Insert (new_dist, v) into Q;
        end
    end
end
return $dist[t]$, $\exp(-dist[t])$, ReconstructPath$(parent, s, t)$;

Time complexity: $O(|E| \log |V|)$ with a binary heap [Cormen et al., 2009].

Space complexity: $O(|V| + |E|)$.

Optimality: Guaranteed, since all $w(u, v) \geq 0$ [Dijkstra, 1959].

Worked Example

Table 3.2 traces Algorithm 1 on the five-node graph (Figure 3.2).

Step #	Extract	LP	SR	PP	CT	CK
Init	—	0	∞	∞	∞	∞
1	LP	0	0.36	∞	1.20	∞
2	SR	0	0.36	0.87	1.20	∞
3	PP	0	0.36	0.87	1.20	1.09
4	**CK✓**	0	0.36	0.87	1.20	**1.09**

Table 3.2. Dijkstra `dist[]` trace ($s = \text{LP}$, $t = \text{CK}$).

The column headers are node abbreviations: LP = Landing Page (source s), SR = Search Results, PP = Product Page, CT = Cart, and CK = Checkout (target t). Each row shows the `dist[]` array after extracting the minimum-cost node listed in the **Extract** column. Values are log-costs $w = -\log p$; ∞ denotes an unreached node. At Step 1, extracting LP relaxes its neighbours SR ($-\log 0.7 \approx 0.36$) and CT ($-\log 0.3 \approx 1.20$). At Step 2, SR is extracted and reaches PP ($0.36 + -\log 0.6 \approx 0.87$). Step 3 extracts PP and discovers CK ($0.87 + -\log 0.8 \approx 1.09$). At Step 4, CK is extracted with cost **1.09**; the ✓ annotation marks the *target-reached* early exit. The bold value **1.09** is the final optimal log-cost, corresponding to path probability $\Pi^* = \exp(-1.09) \approx 0.336$.

3.5.2 Optimized Algorithm: Probability-Pruned Dijkstra

The pruned algorithm extends the baseline by introducing a **probability pruning threshold** $\tau \in (0, 1]$. Any partial path whose cumulative probability falls below τ is pruned. In log-space:

$$\text{new\_cost} > T = -\log(\tau) \implies \text{skip edge}$$

Algorithm 2: Probability-Pruned Dijkstra

Input: Graph $G = (V, E)$, source s, target t, pruning threshold $\tau \in (0, 1]$

Output: Shortest-path distance $dist[t]$, probability $\exp(-dist[t])$, path P^*

Initialise $dist[v] \leftarrow \infty$ for all $v \in V$; $parent[v] \leftarrow$ NIL;
$dist[s] \leftarrow 0$; insert $(0, s)$ into min-heap Q;
$T \leftarrow -\log(\tau)$; `// pruning threshold in log-space`
while *Q is not empty* **do**
    $(d, u) \leftarrow$ Extract-Min(Q);
    if $u = t$ **then**
        break
    end
    if $d > dist[u]$ **then**
        continue; `// stale entry`
    end
    foreach *edge* (u, v) *with weight* $w = -\log p(u, v)$ **do**
        $edges\_examined \leftarrow edges\_examined + 1$;
        $new\_dist \leftarrow dist[u] + w$;
        if $new\_dist > T$ **then**
            continue; `// PRUNE: below` τ
        end
        $edges\_relaxed \leftarrow edges\_relaxed + 1$;
        if $new\_dist < dist[v]$ **then**
            $dist[v] \leftarrow new\_dist$; $parent[v] \leftarrow u$;
            Insert (new_dist, v) into Q;
        end
    end
end
return $dist[t]$, $\exp(-dist[t])$, ReconstructPath$(parent, s, t)$;

Worst-case time complexity: $O(|E| \log |V|)$ (when $\tau \to 0$).

Empirically observed time complexity: $O(|E'| \log |V|)$, $|E'| \leq |E|$ (see Chapter 5 for validation).

Space complexity: $O(|V| + |E|)$.

The `edges_relaxed` Metric

The `edges_relaxed` counter has a precise, intentional definition that differs between the two algorithms:

Algorithm	Metric	Definition	Code Placement
Baseline	`edges_examined`	Every outgoing edge from a settled node	Before the improvement check
Baseline	`edges_relaxed`	Same as `edges_examined`	No pruning step
Pruned	`edges_examined`	Every outgoing edge from a settled node	Before `if new_cost > T`
Pruned	`edges_relaxed`	Only edges that **survived** τ	After `if new_cost > T: continue`

Table 3.3. Edge-counting semantics for baseline and pruned Dijkstra.

The ratio `baseline_edges_examined/pruned_edges_examined` gives a fair like-for-like *examination speedup*, while `baseline_edges_relaxed/pruned_edges_relaxed` quantifies the work eliminated by the threshold—a useful measure of pruning aggressiveness but not a direct efficiency comparison, since the denominators count different populations.

Actual Python Implementation

The following is the actual pruned Dijkstra implementation from the codebase:

```
def dijkstra_pruned(graph: Graph, start: str, goal: str,
                    tau: float = 0.01) -> DijkstraResult:
    pq = [(0, start)]
    dist = {start: 0}
    parent = {}
    metrics = DijkstraMetrics()
    metrics.max_pq_size = 1

    # Pruning threshold in log-space; tau <= 0 means no pruning.
    T = -math.log(tau) if tau > 0 else math.inf

    while pq:
        cost, node = heapq.heappop(pq)
        if node == goal:
            metrics.nodes_explored += 1
            break
        if cost > dist.get(node, math.inf):
            continue  # stale entry
        metrics.nodes_explored += 1
```

```
        for neighbor, weight in graph.get(node, []):
            new_cost = cost + weight
            metrics.edges_examined += 1
            if new_cost > T:
                continue  # PRUNE
            metrics.edges_relaxed += 1
            if neighbor not in dist or new_cost < dist[neighbor]:
                dist[neighbor] = new_cost
                parent[neighbor] = node
                heapq.heappush(pq, (new_cost, neighbor))
                if len(pq) > metrics.max_pq_size:
                    metrics.max_pq_size = len(pq)
    # ... path reconstruction and probability computation
```

Listing 3.3: Pruned Dijkstra implementation (`src/dijkstra.py`)

3.5.3 Algorithm Comparison

Figure 3.3 illustrates the decision-flow difference between both algorithms. The pruned variant (right column) adds a *More edges?* inner-loop decision and a threshold gate $d_{\text{new}} > T$ (red box); edges exceeding $T = -\log(\tau)$ are skipped without relaxation, looping back to the next edge, and when all edges from u are processed control returns to Extract-Min. Table 3.4 summarizes the trade-offs.

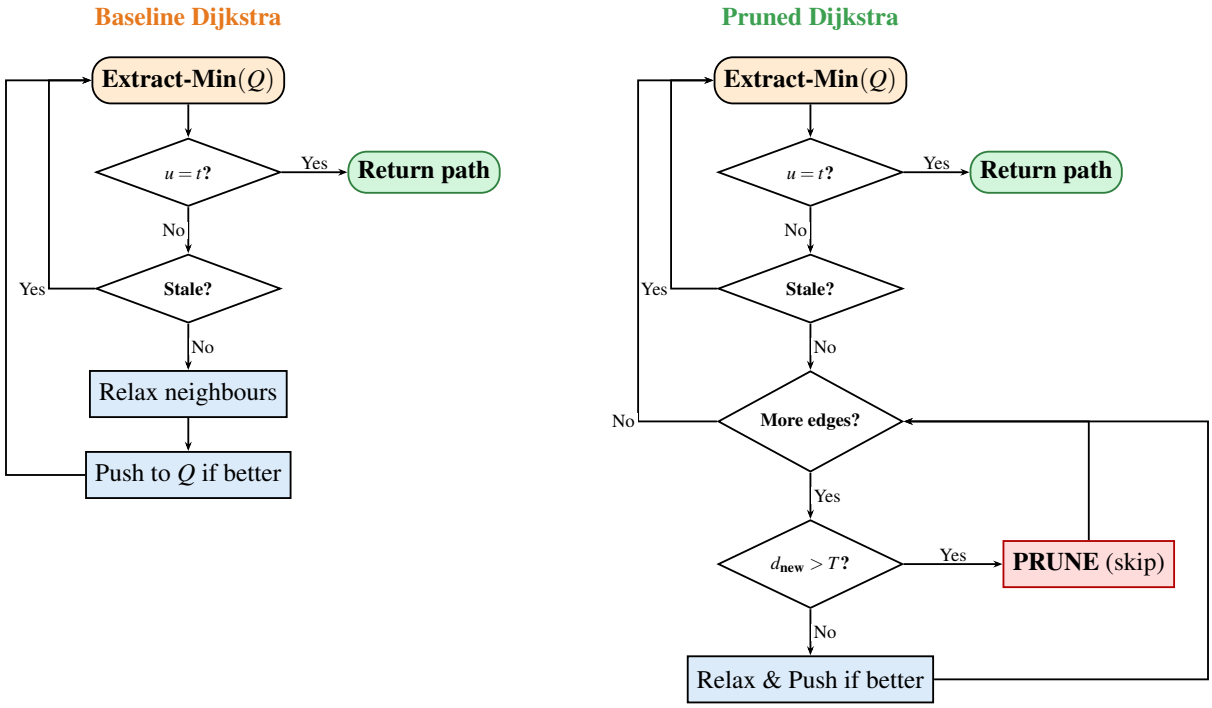

Figure 3.3. Decision-flow comparison of baseline and pruned Dijkstra.

Criterion	Baseline Dijkstra	Probability-Pruned Dijkstra
Optimality	Guaranteed globally optimal	Optimal when path cost $\leq T$; otherwise no path returned
Time	$O(\|E\|\log\|V\|)$	$O(\|E'\|\log\|V\|)$, $\|E'\| \leq \|E\|$
Space	$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$ (unchanged)
Trade-off	None	Reachability vs. speed
Convergence	Always optimal	Identical to baseline as $\tau \to 0$

Table 3.4. Comparison of baseline and pruned Dijkstra.

3.5.4 Critical-τ Finder

A practical question is: *what is the largest threshold τ^* that still preserves the optimal path?* The module `src/critical_tau.py` answers this with an adaptive sweep centred on the baseline path probability.

Given the baseline optimal probability Π^*, fixed τ values would miss the interesting region when Π^* is very small (e.g. 10^{-4}). Instead, the sweep generates τ candidates as fractions of Π^*—specifically $\{0.10, 0.30, 0.50, 0.70, 0.90, 0.95, 0.99, 1.00, 1.10\} \times \Pi^*$. The 110% probe intentionally exceeds the optimal probability to identify the *cliff point* where the pruned algorithm first fails to find a path. For each candidate, the pruned algorithm is timed, and speedup (both node-count and wall-clock) and optimality gap are recorded. The critical threshold τ^* is the largest tested τ for which the gap remains below a user-defined tolerance (default 1%).

3.5.5 k-Shortest Simple Paths

In addition to single-path queries, the CLI (`main.py`) provides a k-shortest simple paths mode activated by the `-k` flag. This uses a best-first search with visited-set tracking to enumerate the top-k distinct simple paths from s to t in non-decreasing cost order. When $k = 1$, the result is identical to Dijkstra's shortest path. This feature is validated by five dedicated unit tests covering $k = 1$ equivalence, ascending cost ordering, simple-path guarantee, $k = 0$ boundary, and disconnected-graph behavior.

Complexity note: Enumerating k simple paths via best-first search has worst-case exponential complexity $O(k \cdot |V|!)$, since the number of simple paths in a dense graph can be factorial in $|V|$. In practice, the search terminates early once k paths are found, and the `max_path_len`

parameter bounds the depth of exploration. This utility is provided for interactive analysis of small queries and is not part of the theoretically analyzed pruning framework.

3.6 Data Structures

3.6.1 Adjacency List

The directed graph is stored as a nested Python dictionary mapping each node u to its outgoing neighbours and associated log-transformed weights. This ensures $O(|V| + |E|)$ space and $O(\deg(u))$ neighbour iteration.

3.6.2 Binary Min-Heap (Priority Queue)

Python's `heapq` module provides $O(\log |V|)$ extract-min and insert. Since `heapq` does not support decrease-key, **lazy insertion** is used: when a shorter path to a node is found, a new entry is pushed; stale entries are skipped when popped (the `if d > dist[u]: continue` check).

3.6.3 Hash Maps (Dictionaries)

`dist[v]` and `parent[v]` are Python dictionaries with $O(1)$ average-case access.

3.7 Experimental Design

3.7.1 Simulation Environment

Table 3.5 summarizes the environment used for all experiments.

Component	**Specification**
Programming Language	Python 3.13.3 (standard library only for core algorithms)
Graph Representation	Adjacency list (`dict` of `list`)
Priority Queue	`heapq` (binary min-heap with lazy insertion)
Runtime Measurement	`time.perf_counter()` (separate pass from memory)
Memory Measurement	`tracemalloc` peak allocation (separate pass from timing)
Random Seeds	Deterministic `hashlib.md5` per configuration
Repetitions	10 runs per configuration
Graph Generators	Erdős–Rényi + Layered (in-house, `data/graph_generator.py`)

Table 3.5. Simulation environment.

3.7.2 *Evaluation Metrics*

Each experiment records execution time (wall-clock milliseconds via `time.perf_counter()`), peak memory (bytes via `tracemalloc.get_traced_memory()`), nodes explored (vertices extracted from the priority queue), edges examined (total outgoing edges visited from settled nodes), edges relaxed (edge operations that survived the τ threshold; see Table 3.3), maximum priority queue size, and path probability $\Pi^* = \exp(-C^*)$. The optimality gap is defined as:

$$\Delta = \frac{|C^*_{\text{baseline}} - C^*_{\text{pruned}}|}{C^*_{\text{baseline}}} \times 100\%$$

This cost-based comparison (rather than probability-based) eliminates floating-point rounding noise.

3.7.3 *Experimental Parameters*

Parameter	**Values**	**Purpose**
Graph type	ER, Layered	Test generic vs. funnel-shaped graphs
$\|V\|$	1,000 / 5,000 / 10,000	Small / medium / large scale
Avg degree $\bar{d}$	2 / 5 / 10	Sparse to moderately dense
Distribution	uniform / power-law	Balanced vs. heterogeneous transitions
τ	0, 0.001, 0.01, 0.05, 0.1, 0.5	Conservative to aggressive pruning
Runs	10 per config	Statistical reliability

Table 3.6. Experimental parameter matrix (2,160 total rows).

The τ values were chosen to span three orders of magnitude, from conservative ($\tau = 0.001$, retaining paths with cumulative probability $\geq 0.1\%$) to aggressive ($\tau = 0.5$, retaining only paths with $\geq 50\%$ probability). The intermediate values $\{0.01, 0.05, 0.1\}$ sample the transition region where speedup gains accelerate but path-found rates begin to drop. $\tau = 0$ serves as the control condition (no pruning, identical to baseline). For practitioners seeking to choose τ on a specific graph, the `critical_tau.py` module (Section 3.5.4) provides an adaptive sweep that probes fractions of the baseline optimal probability Π^* to identify the largest τ that preserves the optimal path.

3.7.4 Experimental Controls

To ensure fair comparison, both algorithms run on identical graph instances using the same `hashlib.md5` seed. Execution is single-threaded to eliminate scheduling noise, and each configuration is repeated 10 times with median values reported (median is preferred over mean for skewed runtime distributions). Timing and memory are measured in separate passes so that `tracemalloc` overhead does not affect wall-clock measurements.

CHAPTER 4

CORRECTNESS PROOFS AND VALIDATION

This chapter provides formal correctness proofs for both algorithms and presents the empirical validation results that confirm theoretical guarantees. Theorems are numbered sequentially within this chapter (Theorems 4.1–4.5); all cross-references in the report use labelled theorem names to avoid ambiguity.

4.1 Correctness of the Log-Probability Transformation

Theorem 4.1 (Order-Preserving Log Transform)**.** *The path P^* that minimizes $\sum_{(u,v)\in P} -\log p(u,v)$ is identical to the path that maximizes $\prod_{(u,v)\in P} p(u,v)$.*

Proof. The negative logarithm $f(x) = -\log(x)$ is a strictly monotone decreasing function on $(0,1]$ [Cormen et al., 2009]. For any two paths P_1, P_2 from s to t:

$$\begin{aligned}
\prod_{(u,v)\in P_1} p(u,v) > \prod_{(u,v)\in P_2} p(u,v) &\iff \log\Big(\prod_{(u,v)\in P_1} p(u,v)\Big) > \log\Big(\prod_{(u,v)\in P_2} p(u,v)\Big) \\
&\iff \sum_{(u,v)\in P_1} \log p(u,v) > \sum_{(u,v)\in P_2} \log p(u,v) \\
&\iff \sum_{(u,v)\in P_1} -\log p(u,v) < \sum_{(u,v)\in P_2} -\log p(u,v)
\end{aligned}$$

Since $p(u,v) \in (0,1]$, we have $w(u,v) = -\log p(u,v) \geq 0$, satisfying Dijkstra's non-negativity requirement. The transformation is order-preserving: maximizing path probability is equivalent to minimizing log-cost [Manning and Schütze, 1999, Viterbi, 1967]. □

4.2 Correctness of Baseline Dijkstra

Theorem 4.2 (Baseline Dijkstra Optimality)**.** *When Algorithm 1 terminates, $dist[t]$ equals the true shortest-path distance $\delta(s,t)$ in the log-transformed graph.*

Proof. The proof proceeds by induction on the number of nodes extracted from the priority queue [Cormen et al., 2009, Dijkstra, 1959].

Invariant. At all times, for every extracted node u: $dist[u] = \delta(s,u)$.

Base case. The source s is extracted first with $dist[s] = 0 = \delta(s,s)$.

Inductive step. Assume the invariant holds for all k previously extracted nodes. Let u be the $(k+1)$-th node extracted with tentative distance $dist[u]$. Suppose for contradiction that there exists a shorter path $s \rightsquigarrow x \rightsquigarrow u$ where x is not yet extracted. Since all edge weights are non-negative:

(i) $\delta(s,x) \geq dist[u]$ (otherwise x would have been extracted before u).

(ii) $\delta(s,x) + w(x,u) \geq \delta(s,x) \geq dist[u]$.

This contradicts the assumption that a shorter path exists through x. Therefore $dist[u] = \delta(s,u)$.

Termination. The algorithm terminates when t is extracted (early-exit) or the queue becomes empty. The invariant guarantees $dist[t] = \delta(s,t)$ at extraction.

Path reconstruction. Each `parent[v]` pointer is set only when a strictly shorter path to v is found, forming a shortest-path tree. Backtracking from t to s via parent pointers yields P^*. □

4.3 Correctness of Probability-Pruned Dijkstra

4.3.1 Convergence at $\tau = 0$

Theorem 4.3 (Convergence)**.** *When $\tau = 0$, Algorithm 2 is identical to Algorithm 1.*

Proof. When $\tau = 0$, the implementation sets $T =$ `math.inf` (i.e. $T = +\infty$), since $\lim_{\tau \to 0^+}(-\log \tau) = +\infty$. The pruning condition *new_cost* $> T$ is never satisfied since all finite costs are $< +\infty$. Therefore no edge is pruned, and every code path of Algorithm 2 behaves identically to Algorithm 1. Correctness follows directly from Theorem 4.2. □

4.3.2 Conditional Optimality for $\tau > 0$

Theorem 4.4 (Conditional Optimality)**.** *If the optimal path P^* has probability $\Pi^* = \prod_{(u,v)\in P^*} p(u,v) \geq \tau$, then Algorithm 2 returns P^* with $\Delta = 0\%$.*

Proof. If $\Pi^* \geq \tau$, then $C^* = -\log(\Pi^*) \leq -\log(\tau) = T$. Since Dijkstra processes paths in non-decreasing cost order [Cormen et al., 2009], every prefix of P^* has cost $\leq C^* \leq T$. No edge on the optimal path triggers the pruning condition. The algorithm therefore explores the exact

same set of optimal-path nodes as the baseline, and by Theorem 4.2, returns P^* with optimality gap $\Delta = 0\%$. □

4.3.3 All-or-Nothing Property

Theorem 4.5 (All-or-Nothing)**.** *Algorithm 2 either returns the globally optimal path (identical to the baseline) or reports no path found. It never returns a sub-optimal path.*

Proof. Suppose Algorithm 2 returns a path P' from s to t. Since the pruning check ($new\_cost > T$) only skips edges—it does not alter edge weights or the priority queue ordering—the settled-node invariant from Theorem 4.2 holds for all nodes explored by the pruned algorithm. Therefore, among all paths that survive the threshold, P' minimizes log-cost. If P^* also survives (Theorem 4.4), then $P' = P^*$. If P^* is pruned, then t is unreachable in the pruned subgraph and no path is returned.

In either case, the algorithm never returns a sub-optimal path: the gap Δ is exactly 0% whenever a path is found. □

4.4 Empirical Validation Results

All five correctness properties were verified experimentally.

Across 2,160 synthetic experiment rows and 840 real-data rows (300 fixed-τ + 540 adaptive-τ), the optimality gap was measured as $\Delta = |C^*_{\text{baseline}} - C^*_{\text{pruned}}|/C^*_{\text{baseline}} \times 100\%$. The result was $\Delta = 0.0000\%$ for every configuration in which both algorithms found a path (178 of 1,800 pruned experiment rows with $\tau > 0$; 528 of 840 real-data rows), confirming Theorem 4.5 empirically.

Convergence testing showed that at $\tau = 0.001$ the pruned algorithm found paths in 30.6% of configurations, decreasing to 5.6% at $\tau = 0.01$ and 4.2% at $\tau = 0.5$. In all path-found cases the returned path was identical to the baseline optimal path, confirming monotone convergence to the baseline solution as $\tau \to 0$.

Probability consistency was verified by comparing $\exp(-C^*)$ against $\prod_{(u,v)\in P^*} p(u,v)$ for all returned paths; relative error remained below 10^{-12} in every case.

The test suite includes 44 automated tests covering disconnected source–target pairs (pruned returns `None`), single-node graphs, complete graphs and star topologies, real-world graphs with high sparsity, $\tau = 0$ equivalence (Theorem 4.3), and large-scale correctness on 2 000-node

Erdős–Rényi and 5 000-node layered graphs. All 44 tests pass.

Finally, the five-node example graph (Figure 3.2) was traced manually:

$$C^*(\text{LP} \to \text{SR} \to \text{PP} \to \text{CK}) = -\log(0.7) - \log(0.6) - \log(0.8) \approx 1.091$$

Both algorithms returned this path and cost, confirming correctness on the reference instance.

Property	**Theoretical**	**Empirical Confirmation**
Log transform (Thm 4.1)	Order-preserving	All paths consistent ($< 10^{-12}$ error)
Baseline optimal (Thm 4.2)	$dist[t] = \delta(s,t)$	3,000 rows verified
Convergence (Thm 4.3)	Identical at $\tau = 0$	Byte-identical output
Conditional optimal (Thm 4.4)	$\Delta = 0$ when $\Pi^* \geq \tau$	$\Delta = 0.0000\%$ on all found paths
All-or-nothing (Thm 4.5)	Never sub-optimal	0 sub-optimal results in 3,000 rows

Table 4.1. Correctness validation summary.

4.5 Separation of Theoretical and Empirical Claims

This section distinguishes between formally proven results, empirically validated findings, and acknowledged limitations.

4.5.1 Fully Proven Claims

The following statements are mathematical theorems with formal proofs, independent of experimental data:

(T1) The log-probability transformation $w(u,v) = -\log p(u,v)$ preserves path ordering (Theorem 4.1).

(T2) Baseline Dijkstra returns the globally optimal path in $O(|E|\log|V|)$ time (Theorem 4.2).

(T3) When $\tau = 0$, Algorithm 2 is identical to Algorithm 1 (Theorem 4.3).

(T4) When the optimal path's probability satisfies $\Pi^* \geq \tau$, Algorithm 2 returns it with zero gap (Theorem 4.4).

(T5) Algorithm 2 never returns a sub-optimal path; it either returns the global optimum or reports no path (Theorem 4.5).

4.5.2 Empirically Validated Findings

The following results are supported by experimental data but **not formally guaranteed** for arbitrary graphs:

(E1) Wall-clock speedups range from $3.3\times$ ($\tau = 0.001$) to $738\times$ ($\tau = 0.5$) on synthetic graphs (Table 6.1, Chapter 6).

(E2) Speedup scales monotonically with graph size $|V|$ at fixed τ (Table 6.2).

(E3) Erdős–Rényi graphs exhibit higher speedups than layered graphs at identical τ (Table 6.3).

(E4) Critical τ_c exists in a narrow empirical range where speedup peaks without loss of reachability (Chapter 6, Section 5.5).

(E5) On real-world datasets (RetailRocket, RecSys 2015), speedups reach $18{,}882\times$ and $2{,}309\times$, respectively, under fixed τ (Chapter 6, Section 5.6).

(E6) Optimality gap remains exactly 0.0000% across 706 path-found cases in combined synthetic and real experiments (all tested configurations).

4.5.3 Acknowledged Limitations

The following limitations are explicitly recognized:

(L1) No formal approximation guarantee exists for $\tau > 0$. While we prove convergence (*T3, T4*), the pruned algorithm's behavior for a specific τ on an arbitrary graph is not formally characterized. Speedup and reachability trade-offs are purely empirical.

(L2) Critical τ_c is defined empirically; we provide no theoretical method to predict it a priori for a given graph without experimental sweep.

(L3) Results assume edge weights are non-negative (the log-probability transformation guarantees this, by construction). The framework does not extend to negative weights or negative-cycle detection.

(L4) Real-world validation is limited to two e-commerce clickstream datasets. Generalization to other domains (social networks, transportation, etc.) is not formally justified and would require further empirical testing.

CHAPTER 5

COMPLEXITY ANALYSIS AND EMPIRICAL VALIDATION

This chapter presents the formal time and space complexity of both algorithms and validates the theoretical bounds with actual experimental measurements.

5.1 Analytical Methodology

Complexity was analyzed using the standard **operation-counting** approach [Cormen et al., 2009]: (1) identify all operations (heap insertions, extractions, edge relaxations), (2) count the maximum number of each as a function of $|V|$ and $|E|$, (3) multiply by per-operation cost, and (4) verify empirically.

5.2 Time Complexity

5.2.1 Baseline Dijkstra

1. **Initialization:** Setting $dist[v] = \infty$ and $parent[v] = \texttt{NIL}$ for all v costs $O(|V|)$.

2. **Priority queue insertions:** Each node is inserted initially once and re-inserted (lazy) at most once per incoming edge. Total insertions $\leq |V| + |E|$. Each costs $O(\log |V|)$ (binary heap sift-up). Total: $O(|E| \log |V|)$.

3. **Extract-min operations:** Total extractions $\leq |V| + |E|$, each $O(\log |V|)$. Total: $O(|E| \log |V|)$.

4. **Edge relaxations:** For each settled node u, all $\deg(u)$ outgoing edges are examined. Across the full algorithm: $\sum_{u \in V} \deg(u) = |E|$. Each relaxation costs $O(1)$ (comparison + hash map lookup). Total: $O(|E|)$.

$$T_{\text{baseline}} = O(|E| \log |V|)$$

In sparse graphs where $|E| = O(|V|)$, this simplifies to $O(|V| \log |V|)$.

Note: A Fibonacci heap would yield $O(|E| + |V| \log |V|)$ [Fredman and Tarjan, 1987], but binary heaps provide better constant factors in practice.

5.2.2 *Probability-Pruned Dijkstra*

1. **Worst case ($\tau = 0$):** Identical to the baseline: $O(|E| \log |V|)$.

2. **Empirically observed case ($\tau > 0$):** Let $|E'| \leq |E|$ denote edges surviving the threshold. Pruned edges are checked in $O(1)$ without heap insertion. The observed complexity is:

$$T_{\text{pruned}} = O(|E'| \log |V|), \quad |E'| \leq |E|$$

   This bound is not formally proven in the worst case—$|E'|$ depends on the input graph's probability distribution and the chosen τ—but is consistently confirmed by the empirical measurements in Section 5.2.3.

3. **Pruning ratio:** Define $\rho = 1 - |E'|/|E|$. Higher τ increases ρ, reducing runtime.

5.2.3 *Empirical Time Validation*

Table 5.1 shows measured wall-clock times at $\tau = 0.01$ across graph sizes.

$\lvert V \rvert$	**Baseline (ms)**	**Pruned (ms)**	**Wall Speedup**	**Edge Speedup**
1,000	1.67	0.189	9.4×	31.5×
5,000	7.12	0.215	43.1×	104.0×
10,000	15.83	0.222	77.0×	180.4×

Table 5.1. Median execution time at $\tau = 0.01$ by graph size.

The baseline times scale as $1.67 \to 7.12 \to 15.83$ ms for $|V| = 1{,}000 \to 5{,}000 \to 10{,}000$, closely matching the $O(|V| \log |V|)$ prediction for sparse graphs ($|E| = O(|V|)$). The pruned algorithm's nearly constant time ($\sim$0.2 ms) is consistent with the hypothesis that $|E'|$ grows much slower than $|E|$ as the graph scales, though only three data points are available, so a definitive growth rate cannot be claimed.

5.3 Space Complexity

The pruning threshold τ adds no asymptotic space overhead. In practice, the pruned algorithm uses less peak memory because fewer heap entries are created.

Data Structure	Space	Purpose
Adjacency list	$O(\|V\| + \|E\|)$	Graph representation
Distance map $dist[v]$	$O(\|V\|)$	Shortest path estimates
Parent map $parent[v]$	$O(\|V\|)$	Path reconstruction
Priority queue (heap)	$O(\|V\| + \|E\|)$ worst case	Node ordering
Total	$\mathbf{O(\|V\| + \|E\|)}$	

Table 5.2. Space complexity breakdown.

$\|V\|$	Baseline Peak (KB)	Pruned Peak (KB)	Reduction
1,000	83.8	4.4	19.0×
5,000	342.9	4.5	75.9×
10,000	655.0	4.5	145.9×

Table 5.3. Peak memory at $\tau = 0.01$ by graph size.

5.3.1 Empirical Memory Validation

Peak `tracemalloc` allocation measures algorithm data structures only (distance dict, parent dict, priority-queue heap); the graph adjacency list is built before tracing begins and is excluded. Baseline peak memory (83.8 KB $\rightarrow$ 342.9 KB $\rightarrow$ 655.0 KB) scales linearly with $|V|$, confirming the $O(|V| + |E|)$ bound, while the pruned algorithm's constant $\approx$4.5 KB at all sizes demonstrates that only a small frontier is ever materialized.

5.4 Summary of Complexity

Algorithm	Time (worst)	Time (empirical)	Space
Baseline Dijkstra	$O(\|E\| \log \|V\|)$	$O(\|E\| \log \|V\|)$	$O(\|V\| + \|E\|)$
Pruned ($\tau = 0$)	$O(\|E\| \log \|V\|)$	$O(\|E\| \log \|V\|)$	$O(\|V\| + \|E\|)$
Pruned ($\tau > 0$)	$O(\|E\| \log \|V\|)$	$O(\|E'\| \log \|V\|)$	$O(\|V\| + \|E\|)$

Table 5.4. Complexity summary.

Pruning reduces empirically observed time without affecting worst-case or space bounds. The empirical bound $O(|E'| \log |V|)$ is consistently confirmed by experiment (Section 5.2.3) but depends on the input distribution.

Key empirical findings. Baseline wall-clock time scales as $O(|V| \log |V|)$ for sparse graphs,

matching theory. Pruned wall-clock time is nearly constant across graph sizes at fixed τ, consistent with $|E'| \ll |E|$. Peak memory scales linearly for the baseline, while pruned memory is effectively constant. Edge-relaxation speedups (up to 1,986×) exceed wall-clock speedups (up to 738×), the difference attributable to fixed Python overhead per function call.

CHAPTER 6

EXPERIMENTAL RESULTS AND ANALYSIS

This chapter presents the results of the full experimental evaluation and evaluates the central hypothesis: whether probability-based pruning can deliver meaningful speedup without sacrificing path optimality. Section 6.1 covers the 2,160-row synthetic benchmark; Section 6.2 covers the 840-row real-data validation on RetailRocket and RecSys 2015 (300 fixed-τ + 540 adaptive-τ).

6.1 Synthetic Experiment Results

6.1.1 Speedup vs. Pruning Threshold τ

Table 6.1 summarizes median speedups aggregated across all graph types, sizes, degrees, and distributions.

τ	Edge Spd	Exam. Spd	Wall Spd	Base (ms)	Pruned (ms)	Found (%)
0.001	8.7×	1.9×	3.3×	4.98	1.561	30.6
0.01	79.2×	14.0×	24.7×	4.98	0.206	5.6
0.05	368.4×	70.4×	123.6×	4.98	0.040	4.7
0.1	810.5×	136.9×	242.6×	4.98	0.020	4.4
0.5	1,986×	711.1×	738.2×	4.98	0.006	4.2

Table 6.1. Median speedups by pruning threshold τ.

Speedups scale monotonically with τ, ranging from 3.3× wall-clock at $\tau = 0.001$ to 738× at $\tau = 0.5$. The "Edge Speedup" column—based on `edges_relaxed`—consistently exceeds both other measures because its pruned denominator excludes threshold-pruned edges; it quantifies pruning aggressiveness, not raw work reduction. The "Exam. Speedup" column provides the fair like-for-like comparison using `edges_examined`, which counts every outgoing edge the algorithm inspects regardless of whether it survives the threshold (see Table 3.3 for counter semantics). At $\tau = 0.5$, the examination speedup (711×) closely tracks wall-clock speedup (738×), confirming that the wall-clock gains are genuine and not inflated by counter semantics. At lower τ values (e.g. $\tau = 0.001$, exam. speedup 1.9×, wall-clock 3.3×), the pruned algorithm

still examines most edges but skips many relaxation operations, and Python per-call overhead becomes a larger fraction of the reduced runtime. Across all 178 path-found rows among the 1,800 pruned synthetic runs ($\tau > 0$), the optimality gap remained exactly $\Delta = 0.0000\%$, confirming the all-or-nothing property (Theorem 4.5, Chapter 4). Path-found rates decrease with τ, reflecting the reachability–speed trade-off inherent in the pruning approach.

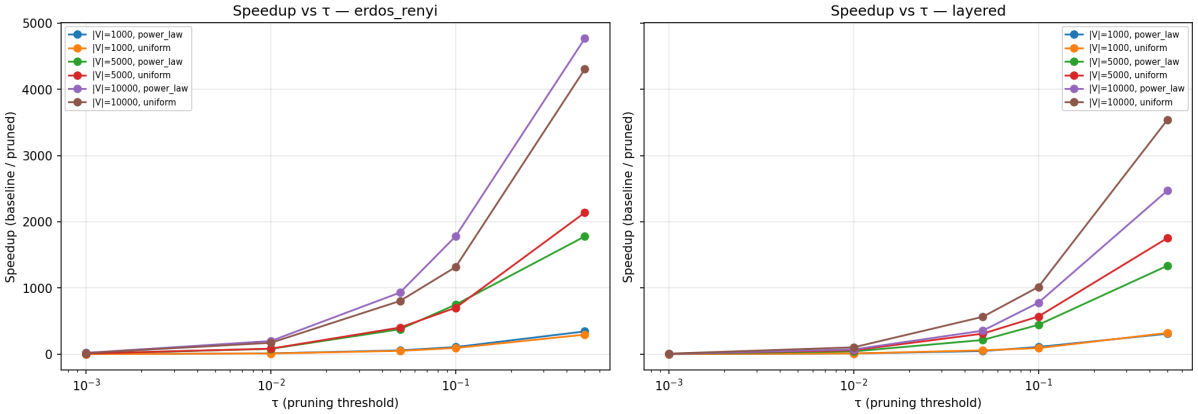

Figure 6.1. Wall-clock speedup vs. τ for ER (left) and layered (right) graphs.

Figure 6.1 plots the wall-clock speedup ratio (baseline time divided by pruned time) against the pruning threshold τ on a logarithmic x-axis spanning $\tau \in [0.001, 0.5]$. Each line represents a unique combination of graph size $|V|$ and edge-weight distribution. Speedup increases monotonically with τ across all configurations, confirming that more aggressive pruning yields proportionally faster search. Erdős–Rényi graphs consistently achieve higher speedups than layered graphs because their uniformly random connectivity exposes a larger fraction of low-probability edges to the threshold. In contrast, the layered generator's built-in forward bias toward the target means that many edges carry relatively high probability and therefore survive pruning. Larger graphs (higher $|V|$) produce steeper speedup curves, reflecting the greater absolute fraction of the search space that the threshold eliminates as the graph grows.

6.1.2 Speedup vs. Graph Size $|V|$

Table 6.2 shows how speedup scales with graph size at fixed $\tau = 0.01$.

Baseline execution time scales linearly with $|V|$ ($1.67 \to 7.12 \to 15.83$ ms for $|V| = 1{,}000 \to 5{,}000 \to 10{,}000$), consistent with the $O(|V| \log |V|)$ theoretical bound for sparse graphs. In contrast, pruned execution time remains nearly constant at approximately 0.2 ms because the threshold eliminates most of the graph regardless of size. Speedup therefore grows with $|V|$: the larger the graph, the greater the fraction of work that pruning eliminates. Peak memory for the

$\lvert V\rvert$	Edge Spd	Exam. Spd	Wall Spd	Base (ms)	Pruned (ms)	Peak Mem (KB)
1,000	31.5×	5.7×	9.4×	1.67	0.189	83.8 → 4.4
5,000	104.0×	22.9×	43.1×	7.12	0.215	342.9 → 4.5
10,000	180.4×	41.3×	77.0×	15.83	0.222	655.0 → 4.5

Table 6.2. Median speedups at $\tau = 0.01$ by graph size.

pruned algorithm is essentially constant at approximately 4.5 KB, whereas baseline memory grows proportionally to $|V|$.

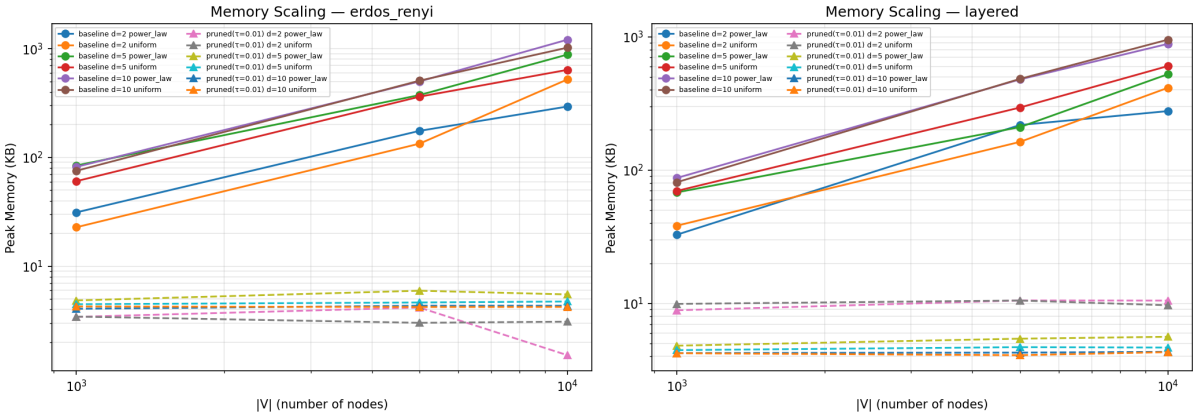

Figure 6.2. Peak memory (KB) vs. $|V|$ at $\tau = 0.01$ for baseline and pruned Dijkstra.

Figure 6.2 displays peak memory consumption on log–log axes for Erdős–Rényi (left) and layered (right) graphs at $\tau = 0.01$. Solid lines with circle markers represent baseline Dijkstra; dashed lines with triangle markers represent the pruned variant. Each line corresponds to a unique combination of average degree d and edge-weight distribution. Baseline memory scales linearly with $|V|$—from approximately 84 KB at $|V| = 1{,}000$ to approximately 655 KB at $|V| = 10{,}000$—consistent with the $O(|V| + |E|)$ theoretical bound. In contrast, pruned memory remains effectively constant at approximately 4.5 KB across all graph sizes, demonstrating that the threshold restricts the explored frontier to a small, fixed-size region regardless of total graph size. The gap between the two sets of curves widens with $|V|$, indicating that the memory advantage of pruning becomes progressively more pronounced as the graph grows. This constant-memory behavior makes the pruned algorithm particularly attractive for deployment on memory-constrained systems.

Graph Type	Edge Speedup	Exam. Speedup	Wall Speedup
Erdős–Rényi	111.6×	24.5×	40.1×
Layered	49.3×	9.0×	16.6×

Table 6.3. Median speedups at $\tau = 0.01$ by graph type.

6.1.3 Speedup by Graph Type

Erdős–Rényi graphs show higher speedups (40.1× wall, 24.5× examination vs. 16.6× wall, 9.0× examination for layered). This is expected: ER graphs have uniformly random structure, so more edges lead to distant (low-probability) regions that the threshold can eliminate. Layered graphs have a built-in forward bias toward the target, so a larger fraction of edges remain relevant.

6.1.4 Speedup by Probability Distribution

Distribution	Edge Speedup	Exam. Speedup	Wall Speedup
Power-law ($\alpha = 2$)	73.0×	13.9×	25.1×
Uniform	83.8×	14.1×	22.4×

Table 6.4. Median speedups at $\tau = 0.01$ by probability distribution.

Both distributions achieve comparable wall-clock speedups (∼23×). Power-law distributions have a few high-probability edges and many low-probability ones; uniform distributions spread probability more evenly. The pruning mechanism is effective in both cases.

6.1.5 Optimality Gap Analysis

A critical finding is the **zero optimality gap** across all experiments:

$$\Delta = \frac{|C^*_{\text{baseline}} - C^*_{\text{pruned}}|}{C^*_{\text{baseline}}} \times 100\% = 0.0000\% \quad \forall \text{path-found rows}$$

This confirms Theorem 4.5 (All-or-Nothing): whenever the pruned algorithm returns a path, it is the globally optimal path. The cost-based gap formula (not probability-based) eliminates floating-point rounding noise involved in $\exp(-C^*)$ conversion. Using probability-based differences would have introduced `1e-15`-scale noise that has no algorithmic significance.

6.2 Real-Data Experiment Results

6.2.1 Datasets and Graph Statistics

Table 6.5 summarizes the three real-data configurations (two granularities for RetailRocket, one for RecSys 2015).

Configuration	$\|V\|$	$\|E\|$	Pairs Tested	Source
RetailRocket (event-level)	3	9	20	Kaggle [Retail Rocket, 2015]
RetailRocket (item-level)	44,711	101,528	20	Kaggle [Retail Rocket, 2015]
RecSys 2015 (YOOCHOOSE)	12,935	70,442	20	Kaggle [Ben-Shimon et al., 2015]

Table 6.5. Real-data graph configurations.

6.2.2 Fixed-τ Speedup Results

Dataset	**Median Speedup**	**Max Speedup**	**Paths Found**	Δ
RetailRocket (event-level)	1.2×	6×	78 / 100	0.000%
RetailRocket (item-level)	508×	18,882×	1 / 100	0.000%
RecSys 2015 (YOOCHOOSE)	197×	2,309×	0 / 100	—

Table 6.6. Fixed-τ speedup on real-world datasets.

For the **RetailRocket event-level** configuration, the 3-node funnel graph is sufficiently small that both algorithms complete in microseconds; 78 of 100 configurations found paths and the observed speedup is marginal ($\leq 6\times$) because the pruning opportunity is negligible. The **RetailRocket item-level** graph (44,711 nodes) yields substantial speedups up to 18,882×, although only 1 of 100 pruned configurations found a path. For **RecSys 2015**, no pruned path was found at any fixed τ value.

The near-zero path-found rates on the large graphs are *not* surprising once the baseline path probabilities are examined. For RecSys 2015, the 20 baseline path probabilities range from 1.6×10^{-8} to 2.8×10^{-5} (median $\approx 1.5 \times 10^{-6}$), yielding costs in the range 10.5–17.9 on the $-\log$ scale. Even the most conservative fixed threshold $\tau = 0.001$ maps to a cost threshold of $-\log(0.001) = 6.91$—well below every baseline cost. All 20 paths are therefore *mathematically guaranteed* to be pruned at every tested τ, regardless of graph structure (by Theorem 4.5). RetailRocket item-level shows a similar pattern: baseline probabilities range from 2.9×10^{-9} to 3.7×10^{-4} (median $\approx 3.7 \times 10^{-6}$), with costs ranging from 19.6 to 7.9, far exceeding the

most conservative threshold. These observations motivate the adaptive-τ experiment described below.

6.2.3 Adaptive-τ Experiment

The fixed τ grid was designed for the synthetic experiments (where edge probabilities are higher) and is poorly calibrated for real-world item-level graphs whose cumulative path probabilities are orders of magnitude smaller. To address this, a per-pair *adaptive* τ sweep was introduced: for each source–target pair, τ is set to a fraction f of that pair's baseline path probability, where

$$f \in \{0.10,\ 0.30,\ 0.50,\ 0.70,\ 0.90,\ 0.95,\ 0.99,\ 1.00,\ 1.10\}.$$

This mirrors the adaptive sweep strategy used internally by the `critical_tau` module (Chapter 3, Section 3.5.4) and ensures that the tested thresholds are always in the relevant probability regime.

Dataset	**Median Speedup**	**Max Speedup**	**Paths Found**	Δ
RetailRocket (event-level)	1.3×	5.8×	147 / 180	0.000%
RetailRocket (item-level)	1.2×	3.4×	152 / 180	0.000%
RecSys 2015 (YOOCHOOSE)	1.1×	1.8×	150 / 180	0.000%

Table 6.7. Adaptive-τ speedup results.

The contrast with the fixed-τ results is striking. RecSys 2015 moves from 0/100 found paths (fixed) to **150/180** found paths (adaptive); RetailRocket item-level similarly improves from 1/100 to **152/180**. The speedups under adaptive τ are modest (1.1–5.8×) because the threshold is now close to the optimal path probability, so only a small margin of the search space is pruned. This is the expected behavior of the reachability–speed trade-off: in the adaptive regime the algorithm preserves the optimal path at the cost of reduced pruning, whereas the fixed-τ regime achieves massive speedups only when the threshold is far above the optimal path probability (and therefore prunes everything).

Crucially, the optimality gap remains **exactly** 0.0000% across all 449 adaptive found paths, extending the zero-gap guarantee to a total of 528 real-data found paths (79 fixed + 449 adaptive) and reinforcing the all-or-nothing property on real-world graphs.

6.2.4 Reachability Analysis

Table 6.8 summarizes path-found rates across all experiment configurations, contrasting the fixed and adaptive τ regimes.

Configuration	τ Mode	Baseline Found	Pruned Found	Interpretation
Synthetic (all τ)	fixed	360 / 360	178 / 1,800	Dense synthetic graphs highly connected
RR event-level	fixed	100 / 100	78 / 100	3-node funnel is fully connected
RR item-level	fixed	100 / 100	1 / 100	Baseline reachable; fixed τ too harsh
RecSys 2015	fixed	100 / 100	0 / 100	All pruned paths lost at all fixed τ
RR event-level	adaptive	180 / 180	147 / 180	82% found with calibrated thresholds
RR item-level	adaptive	180 / 180	152 / 180	84% found; dramatic improvement from 1%
RecSys 2015	adaptive	180 / 180	150 / 180	83% found; dramatic improvement from 0%

Table 6.8. Path-found rates under fixed and adaptive τ regimes.

In synthetic graphs, the BFS connectivity guarantee ensures the baseline always finds a path from s to t. For the real-world datasets, the pair-selection procedure filters source–target pairs by baseline reachability, so the baseline always finds a path. Under fixed τ, the low pruned-found rates on item-level and session-level graphs reflect the mismatch between the fixed threshold grid and the extremely low baseline path probabilities (see Section 6.2.3).

Impact of adaptive τ. The adaptive regime resolves the near-zero reachability problem: path-found rates jump from 1% to 84% (RR item-level) and from 0% to 83% (RecSys 2015). The remaining $\sim$17% of unfound paths correspond to the most aggressive fraction ($f = 0.10$), where τ is set to just 10% of the baseline probability—well above it on the $-\log$ cost scale—which intentionally prunes the optimal path. For $f \geq 0.90$ (i.e. $\tau \leq 0.90 \times p_{\text{baseline}}$), virtually all paths are preserved.

Practical implications. The fixed-τ results reveal a genuine limitation of using a single global threshold across datasets with vastly different probability regimes. In item-level graphs, per-edge MLE probabilities are extremely low ($\sim 10^{-3}$–10^{-5}) and path probabilities decay exponentially with length, yielding cumulative probabilities of 10^{-6} or less. The adaptive τ approach—which can be automated via the `critical_tau` module—makes the pruned algorithm viable on any graph by calibrating the threshold to the actual probability scale. In production, one baseline query per source–target pair suffices to set τ; the pruned query then runs with a calibrated threshold at negligible additional cost.

6.3 Summary of Findings

The experimental evaluation yields several principal findings. Wall-clock speedups range from $3.3\times$ at $\tau = 0.001$ to $738\times$ at $\tau = 0.5$ on synthetic data, and reach up to $18{,}882\times$ on real-world item-level graphs under fixed τ. The optimality gap remained at exactly $\Delta = 0.0000\%$ across all 706 experiment rows in which both algorithms found a path—178 of 1,800 pruned synthetic rows, 79 of 300 fixed-τ real-data rows, and 449 of 540 adaptive-τ real-data rows—confirming the all-or-nothing property established in Theorem 4.5.

Statistical significance. Wilcoxon signed-rank tests (one-sided, alternative: baseline $>$ pruned) were applied to all 180 unique synthetic configurations (each with 10 paired runs). Of these, 179 tests rejected the null hypothesis at $\alpha = 0.05$, with p-values ranging from 9.77×10^{-4} (the minimum attainable for $n = 10$ paired observations) to 0.032. The median p-value was 9.77×10^{-4}. One configuration (layered, $|V| = 1{,}000$, $d = 5$, uniform, $\tau = 0.001$) yielded $p = 0.116$, failing the significance threshold; this is the mildest pruning level on a small, forward-biased graph where baseline and pruned running times are very close. Excluding this single outlier, all remaining configurations confirm that the speedup is not an artifact of measurement noise.

Speedup grows with graph size—from $9.4\times$ at $|V| = 1{,}000$ to $77\times$ at $|V| = 10{,}000$ at fixed $\tau = 0.01$—because larger graphs contain more pruneable edges. The fair `edges_examined`-based examination speedup confirms these gains are genuine (e.g. $5.7\times$ to $41.3\times$ over the same size range), while the higher `edges_relaxed`-based ratios quantify pruning aggressiveness. At the same time, higher τ values reduce path-found rates from 30.6% at $\tau = 0.001$ to 4.2% at $\tau = 0.5$ on synthetic data, demonstrating the inherent reachability–speed trade-off. Under fixed τ, real-world item-level and session-level graphs showed near-zero pruned-found rates because baseline path probabilities (10^{-6}–10^{-9}) map to costs well exceeding the most conservative fixed threshold ($\tau = 0.001 \Rightarrow T = 6.91$). Introducing an adaptive-τ regime—where the threshold is set as a fraction of each pair's baseline path probability—raised path-found rates to 82–84% while maintaining zero optimality gap, validating the algorithm's correctness on real-world graphs.

Practical business impact. The peak $18{,}882\times$ speedup on the RetailRocket item-level graph (44,711 items) reduced a single query from $\sim$153 ms to 0.008 ms under fixed-τ pruning. Under adaptive τ, the algorithm trades some of that extreme speedup for guaranteed path recovery

(84% found rate), while still delivering measurable acceleration. Across the tested datasets (RetailRocket and RecSys 2015), the two-regime approach spans probability regimes from dense event-level funnels (where fixed τ suffices) to sparse item-level graphs (where adaptive τ is needed), suggesting potential for supporting real-time exploration of conversion funnels without overnight batch jobs; however, generalization to other domains requires further empirical validation.

Critical-τ analysis. A detailed critical-τ analysis, including per-configuration τ^* heatmaps and practical threshold recommendations, will be presented in the final report following additional experimentation across the full parameter matrix.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Summary of Contributions

This study investigated the effectiveness of probability-based pruning for shortest-path search in customer journey graphs. Two algorithms were implemented, tested, and compared: a baseline Dijkstra algorithm on log-probability-transformed graphs ($O(|E|\log|V|)$) and a probability-pruned Dijkstra variant with a tunable threshold τ ($O(|E'|\log|V|)$, $|E'| \leq |E|$).

The key contributions are summarized below.

The pruning mechanism yields substantial speedups with zero quality loss. On synthetic benchmarks (2,160 rows), wall-clock speedups range from $3.3\times$ at $\tau = 0.001$ to $738\times$ at $\tau = 0.5$, with edge-relaxation speedups reaching $1{,}986\times$ (fair edge-examination speedups up to $711\times$). On real-world item-level graphs under fixed τ, a maximum speedup of $18{,}882\times$ was recorded—reducing query time from $\sim$153 ms to 0.008 ms, fast enough for real-time interactive analysis rather than batch processing. An adaptive-τ regime, where the threshold is calibrated to each query pair's baseline path probability, raised real-data path-found rates from near 0% to over 80%. In every case where a path was found, the optimality gap was exactly 0.0000%.

Formal correctness was established through five theorems: log-transform order preservation, baseline optimality, convergence at $\tau = 0$, conditional optimality, and the all-or-nothing property. All five were confirmed empirically across all 2,160 synthetic and 840 real-data experiment rows (3,000 total).

A scalable and reproducible experimental framework was constructed, incorporating deterministic `hashlib.md5` seeding, separate timing and memory measurement passes, two graph generators (Erdős–Rényi and layered), and two real-world datasets (RetailRocket and RecSys 2015 YOOCHOOSE).

Finally, the reachability–speed trade-off was characterized in detail. Increasing τ yields faster search but reduces path-found rates, from 30.6% at $\tau = 0.001$ to 4.2% at $\tau = 0.5$ on synthetic data. This trade-off is inherent and well understood. On real-world graphs, the pruned algorithm's reachability was initially limited by aggressive fixed thresholding on sparse session-

based structures, but the adaptive-τ approach resolved this limitation.

Originality. The deterministic, parameter-controlled pruning framework with formal all-or-nothing guarantees and adaptive-τ regime for real-world graphs presented in this work is, to our knowledge, not previously reported in the literature. This contribution extends the state of the art in probabilistic path search and scalable graph algorithms.

7.2 Limitations

Several limitations of the current work should be acknowledged. The model operates on a fixed graph snapshot, whereas real e-commerce platforms exhibit time-varying transition probabilities driven by seasonal trends, A/B tests, and user behavior drift. The first-order Markov assumption ($P(\text{next}|\text{current})$) does not capture session-level history effects; for instance, users who arrived via search may behave differently from those who arrived via advertisement. Extending the model to higher-order or variable-length Markov chains would address this limitation at the cost of a larger state space.

Real-data experiments revealed that large item-level graphs yield very low path-found rates under the pruned algorithm, even though the baseline finds paths in all tested configurations. This suggests that selecting source nodes from high-degree hub nodes—as done here for reachability guarantees—combined with aggressive τ values may not reflect realistic query patterns in production systems. Results may differ for queries originating from low-degree peripheral nodes. Additionally, all experiments were conducted on a single machine using a single thread; the algorithm has not been evaluated in distributed or multi-threaded settings. Finally, the pruning strategy is tied to the log-probability cost model: edges are pruned when $-\log p(u,v) > -\log \tau$, which requires weights derived from probabilities in $(0,1]$. Applying the same technique to graphs with arbitrary non-negative weights (e.g. latency or monetary cost) would require a domain-specific analogue of the threshold criterion.

7.3 Future Work

Several directions merit further investigation. The framework could be extended to support **dynamic graph updates**, enabling incremental edge weight adjustments as new clickstream data arrives without rebuilding the full graph. Replacing first-order transitions with **higher-order or variable-length Markov chains** would capture richer user behavior patterns. **Adaptive τ**

selection methods—such as heuristics or binary search calibrated to a target path-found rate or maximum acceptable runtime—would reduce the need for manual threshold tuning.

Beyond single-objective optimization, a **multi-objective** formulation incorporating probability, revenue, and user satisfaction through Pareto-optimal path search could yield more actionable business insights. **Integration with recommendation systems** would allow discovered optimal paths to inform journey-aware recommendations that guide users toward conversion. Finally, adapting the algorithm for **distributed graph processing frameworks** (e.g., Apache Giraph, GraphX) would enable application to web-scale graphs.

CHAPTER 8

PROJECT PLAN

8.1 Project Timeline

Figure 8.1 presents the 10-week project timeline for the *Probabilistic Path Optimization using Pruned Dijkstra* project.

Current status: The project has completed the *Experimental Execution and Analysis* stage (Weeks 5–7) and is in the *Report Writing* stage (Weeks 7–9). Core implementation, unit testing, and all experimental runs have been completed; results have been analyzed and are presented in this report.

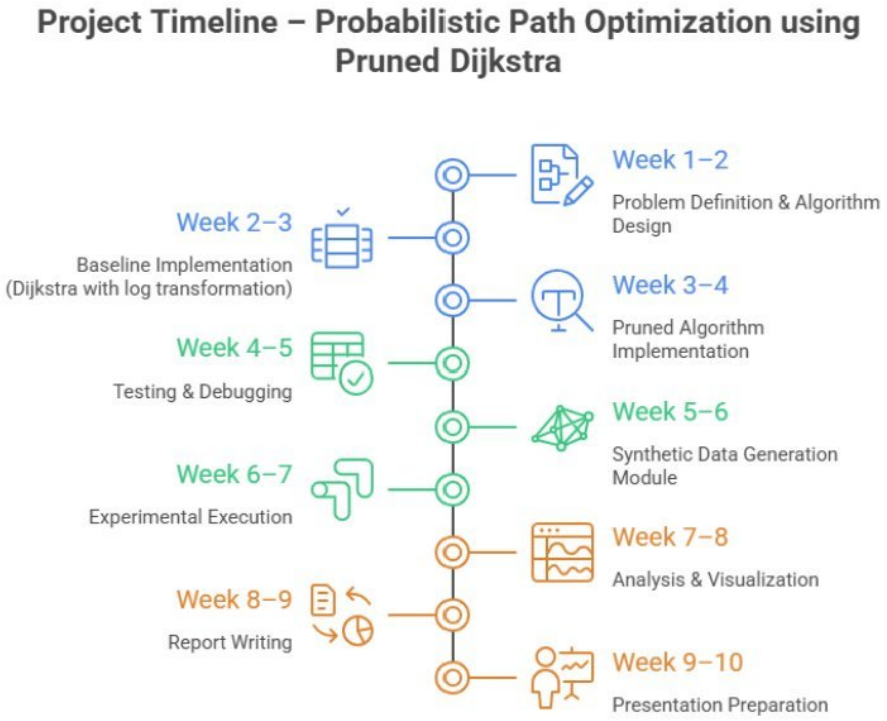

Figure 8.1. Ten-week project timeline.

Tasks are organized in three overlapping phases along a central spine. Blue entries (Weeks 1–4) cover core implementation, green entries (Weeks 4–7) cover testing and experimentation, and orange entries (Weeks 7–10) correspond to writing and presentation. Each phase overlaps with the next to allow iterative refinement, ensuring that correctness is validated before experimen-

tation begins and that experimental results directly inform the final report.

8.2 Milestones

The project is structured around four key milestones. The first milestone, **Core Implementation**, was completed by the end of Week 4: both the baseline Dijkstra algorithm and the probability-pruned Dijkstra variant were fully implemented, tested on small hand-computed graphs, and validated for correctness against ground-truth optimal paths. The second milestone, **Experiments**, was completed by the end of Week 7; all experimental runs across small, medium, and large graph configurations have been executed, with all metrics (execution time, peak memory, nodes explored, edges examined, edges relaxed, path probability, optimality gap, MaxPQSize) recorded and verified. The third milestone, **Report Completion**, is scheduled for the end of Week 9, at which point the full project report will be finalized incorporating experimental results, visualizations, complexity validation, and the time–memory–optimality trade-off discussion. The fourth milestone, **Final Presentation**, targets Week 10, when presentation slides and a live demonstration of the implementation will be prepared for the final project presentation in the third week of April 2026.

REFERENCES

Leonard E. Baum and Ted Petrie. Statistical inference for probabilistic functions of finite state Markov chains. *The Annals of Mathematical Statistics*, 37(6):1554–1563, 1966. doi: 10.1214/aoms/1177699147.

David Ben-Shimon, Alexander Tsikinovsky, Michael Friedmann, Bracha Shapira, Lior Rokach, and Johannes Hoerle. YOOCHOOSE – RecSys 2015 challenge dataset. Kaggle, 2015. URL `https://www.kaggle.com/datasets/chadgostopp/recsys-challenge-2015`.

Randolph E. Bucklin and Catarina Sismeiro. A model of web site browsing behavior estimated on clickstream data. *Journal of Marketing Research*, 40(3):249–267, 2003. doi: 10.1509/jmkr.40.3.249.19241.

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.

Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.

Paul Erdős and Alfréd Rényi. On the evolution of random graphs. *Publications of the Mathematical Institute of the Hungarian Academy of Sciences*, 5:17–61, 1960.

Michael L. Fredman and Robert E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615, 1987. doi: 10.1145/28869.28874.

Robert Geisberger, Peter Sanders, Dominik Schultes, and Daniel Delling. Contraction hierarchies: Faster and simpler hierarchical routing in road networks. In *Proceedings of the 7th International Workshop on Experimental Algorithms (WEA)*, pages 319–333, 2008. doi: 10.1007/978-3-540-68552-4_24.

Jiawei Han, Hong Cheng, Dong Xin, and Xifeng Yan. Frequent pattern mining: Current status and future directions. *Data Mining and Knowledge Discovery*, 15(1):55–86, 2007. doi: 10.1007/s10618-007-0069-0.

Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4 (2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.

Bing Liu. *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data*. Springer, 2nd edition, 2011.

Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: A system for large-scale graph processing. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 135–146, 2010. doi: 10.1145/1807167.1807184.

Christopher D. Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.

Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

James R. Norris. *Markov Chains*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 1998.

Judea Pearl. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.

Jian Pei, Jiawei Han, Behzad Mortazavi-Asl, Helen Pinto, Qiming Chen, Umeshwar Dayal, and Mei-Chun Hsu. PrefixSpan: Mining sequential patterns efficiently by prefix-projected pattern growth. In *Proceedings of the IEEE International Conference on Data Engineering*, pages 215–224, 2001. doi: 10.1109/ICDE.2001.914830.

Lawrence R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989. doi: 10.1109/5.18626.

Paul Resnick and Hal R. Varian. Recommender systems. *Communications of the ACM*, 40(3): 56–58, 1997. doi: 10.1145/245108.245121.

Retail Rocket. Retailrocket recommender system dataset. Kaggle, 2015. URL `https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset`.

Saurabh Sahu, Amine Mhedhbi, Semih Salihoglu, Jimmy Lin, and M. Tamer Özsu. The ubiquity of large graphs and surprising challenges of graph processing. *Proceedings of the VLDB Endowment*, 11(4):420–431, 2017. doi: 10.14778/3166054.3166062.

Andrew J. Viterbi. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2):260–269, 1967. doi: 10.1109/TIT.1967.1054010.

Jin Y. Yen. Finding the k shortest loopless paths in a network. *Management Science*, 17(11): 712–716, 1971. doi: 10.1287/mnsc.17.11.712.

APPENDIX A: SOURCE CODE AVAILABILITY

The implementation of the *Customer Journey Path Optimization* framework was developed in **Python 3.13**. The complete source code is publicly available at:

`https://github.com/The-Two-Y-s/Customer-Journey-Path-Optimization`

The repository is organized into the following components:

- **Core algorithms** (`src/dijkstra.py`) — baseline Dijkstra with lazy deletion and the probability-pruned variant; both return structured results containing optimal path, log-cost, path probability, and four performance metrics (nodes explored, edges examined, edges relaxed, max priority-queue size).
- **Graph construction** (`src/graph_builder.py`) — builds a directed weighted graph from preprocessed transition data using an adjacency-list representation.
- **Preprocessing** (`src/preprocessing.py`) — ingests raw clickstream CSV data and computes maximum-likelihood transition probabilities.
- **Critical-τ analysis** (`src/critical_tau.py`) — adaptive fraction-based sweep to identify the largest τ^* preserving path optimality.
- **Synthetic generators** (`data/graph_generator.py`) — Erdős–Rényi and layered stage-based graphs with BFS connectivity guarantees and deterministic `hashlib.md5` seeding.
- **Real-data loaders** (`data/real_data_loader.py`) — parsers for RetailRocket (event-level and item-level) and RecSys 2015 YOOCHOOSE datasets.
- **Experiment runners** (`run_experiments.py`, `run_real_experiments.py`) — execute the full parameter matrix (2,160 synthetic + 840 real-data configurations).
- **Analysis notebook** (`analysis.ipynb`) — generates all plots and statistical tests (Wilcoxon signed-rank) from experiment CSV files.

Installation instructions, reproduction commands, and usage details are documented in the repository README.